FCMA203 逻辑与计组 **Lab 4 Report** **Fall 2026**

Deadline of submission: 23:59:59, Nov. 1st (Sunday), 2026

Name : 邓淳天 .Stu. ID : SUAT25000185 .

Lab Date : 10/22/2026 . Lab Time : 13:30-14:55 .

必做任务 1: 证明数值大小的判断逻辑

请写出证明过程。

设 A, B 是 n 位二进制数，此处 n 取 32

对于 A-B，可以通过对 B 取补码得到~B+1，对应无符号数值大小为 2^n-B

因此 A-B=A+~B+1=A-B+2^n

当 A, B 为无符号数时

1. A > B 时，A-B>0，则 A-B+2^n 溢出，c_out 为 1

2. A < B 时，A-B<0，则 A-B+2^n 不会溢出，c_out 为 0。

当 A, B 为有符号数时

1. A, B 符号相同时，A[31]^B[31]=0，此时 A B 比较结果 = 0 ^ c_out = c_out，由于 A, B 都使用补码表示，符号相同的情况下，绝对值越大，对应的无符号值也越大，因此此时 c_out 的产生规则与 A B 都为无符号数的情况相同，故 c_out=0 时，A<B，此时 A[31]^B[31]^c_out=0

2. A, B 符号不同时，A[31]^B[31]=1，A B 比较结果=1 ^ c_out = ~c_out。假设 A<B，则 A 为负数，B 为正数，而负数的无符号值>正数的无符号值，故 c_out=1，A<B = (~c_out == 0) == 0，成立；假设 A>B，则反之 A 为正数，B 为负数，c_out==0，A<B = (~c_out == 0) == 1，成立。故有符号数的比较可通过 A[31]^B[31]^c_out 来查看

必做任务 4: testbench 仿真

请将所有 case 都通过仿真的截图附上。

```
source tb_top.tcl
# set curr_wave [current_wave_config]
# if { [string length $curr_wave] == 0 } {
#   if { [llength [get_objects]] > 0} {
#     add_wave /
#     set_property needs_save false [current_wave_config]
#   } else {
#      send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a
#   }
# }
# run 1000ns
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_top_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:10 ; elapsed = 00:00:07 . Memory (MB): peak = 7848.543 ; gain = 137.805 ; free physical = 7
run all
================================================
checks = 7046, errors = 0
TEST PASSED !
All test stimuli passed.
================================================
$finish called at time : 3176 ns : File "/home/blossilver/SUAT/SUAT-Microelectro/Digital-Logic/Lab/Lab4_alu_rf/rtl/test/tb_top.v" Li
```

Reflection: 基于你本次可能并不顺利的实验经历写下你的思考（300 字以内）：

你可以思考（但不限于）以下问题：

- 结果是否与你预期的一致？如果不一致，原因是什么？
- 你是如何 debug 的？
- 是否有对编码 Verilog 的一些好建议？
- ……

一开始运行提供的 tb_top 并没有一次性通过，原因是出在几条 br 指令的检查上。Debug 时，我首先通过 tb_top 的输出，定位问题在几条 br 指令上。首先我检查了 alu_ctrl 的解码，看我是否在对 alu_ctrl 信号的解码上与讲义的说明有不一致的情况。排除了解码问题后，我又通过观察 tb_top 的输出，发现主要是几条易错点 br 指令——blt bltu 上，之后根据讲义对这几 branch 指令的 rtl 实现的讲解，核对我自己的 rtl 实现，定位到问题为有符号数的比较中，A < B 的情况应当为 A[31]^B[31]^c_out 的取反，而我没有取反。

在进行 decode 时，推荐使用宏或者 sv 的 package 先对各个命令对应的 opcode funct3 funct7 部分写好对应的宏或者 typedef enum，如果使用 enum 还会为波形的观察带来很大的的方便，可以直接查看 enum 值的名字，而不是只能看到一串二进制值。